ETHIOSEC SCANNER

AI-Powered Web & API Vulnerability Scanner

"Scan. Understand. Secure."

INSA CYBERSECURITY PROJECT REPORT

Team Member	CTC Number
Abel Biruktesfa	CTC-1481-26
Abel Tilahun	CTC-1696-26
Miftah Hussen	CTC-4008-26
Mikias Getnet	CTC-513-26
Maraki	CTC-2443-26

Information Network Security Administration (INSA)
Cybersecurity Project
October 2026

Acknowledgment

We would like to express our sincere appreciation to the Information Network Security Administration (INSA), our instructors, mentors, and everyone who supported our cybersecurity learning journey and project development. The training and guidance provided us with a valuable opportunity to connect cybersecurity concepts with practical software development and responsible security assessment.

We would like to extend our special gratitude to our instructor and mentor, **Mr. Fasika Mekbib**, for his incredible guidance, continuous support, and valuable insights, not only throughout the cybersecurity summer camp but also during every stage of our project development. His dedication, encouragement, and willingness to guide us through challenges have played a significant role in our learning and progress.

We are also deeply thankful to the web development students who collaborated with us and contributed to the web development aspects of our project. We sincerely appreciate their time, constructive feedback, and honest opinions, especially during the testing phase. Their willingness to share their genuine thoughts and point out areas for improvement helped us evaluate our work from different perspectives and improve the quality of our project.

Finally, we extend our heartfelt thanks to everyone who contributed their time, knowledge, feedback, and support throughout this journey. Every contribution, whether large or small, has been valuable to our progress. We are truly grateful to all those who stood by us, helped us overcome challenges, and made this project development experience a meaningful learning opportunity.

1. Introduction 5
1.1 Background 5
1.2 Project Overview 5
2. Problem Statement 5
3. Objectives 6
3.1 General Objective 6
3.2 Specific Objectives 6
4. Scope and Limitations 6
4.1 Scope 6
4.2 Limitations 6
5. Methodology and System Architecture 7
5.1 Development Methodology 7
5.2 System Architecture 7
Layer 8
Technologies 8
Frontend 8
Vanilla HTML5, CSS3, JavaScript (ES6+), Modern Cybersecurity Theme 8
Backend API 8
Python 3.11+, FastAPI, Starlette, Uvicorn 8
Database 8
SQLAlchemy 2.0 ORM (PostgreSQL in production, SQLite for local dev) 8
Authentication 8
Argon2id password hashing, Stateless JWT (HS256) with centralized session revocation, 6-digit email verification 8
Scanning Engine 8
Asynchronous HTTP crawler, differential analysis, timing-attack verification, heuristic payload injection 8
AI Analysis 8
Multi-provider AI module (Google Gemini API support) 8
The general workflow of the system is:1. The user enters an authorized target URL.2. The system validates the provided input.3. The backend starts the scanning process.4. The scanner performs the selected security checks.5. The system analyzes the collected information.6. Detected issues are organized into findings.7. The results and recommendations are presented to the user through the frontend. 8
6. System Implementation 9
7. Security and Ethical Considerations 9
8. Testing and Results 10

9. Challenges and Solutions 12
10. Discussion 13
11. Future Improvements 13
12. Conclusion 14
13. References 15

1. Introduction

1.1 Background

Websites and web applications are widely used for communication, education, business, and many other services. As their use increases, security has become an important concern because vulnerabilities can put systems and information at risk.
Finding these security issues manually can be difficult and time-consuming. For this reason, our team developed EthioSec Scanner, a web and API vulnerability scanner designed to automatically identify common security weaknesses in authorized targets. Its main purpose is to make vulnerability scanning easier by detecting security issues, analyzing the findings, and providing clear explanations and recommendations to help users understand and fix the identified problems

1.2 Project Overview

EthioSec Scanner is a web and API vulnerability scanning platform developed to help users identify common security weaknesses in authorized websites and applications. The system scans a given target, analyzes different security configurations and potential vulnerabilities, and presents the detected issues in an understandable format.
The platform focuses on areas such as security headers, cookies, CORS configuration, server information disclosure, and other web security checks. It is also designed to provide useful explanations and recommendations for detected issues, helping users understand the problem and how it can be improved.
The project combines cybersecurity concepts with web development and automation to provide a simple and practical security assessment platform.

2. Problem Statement

As websites and web applications become more common, security weaknesses such as poor configurations, missing security headers, insecure cookies, and other vulnerabilities can create risks for both users and system owners. However, identifying these problems manually can be difficult and time-consuming, especially for developers and users who have limited cybersecurity experience.

Although different security scanning tools already exist, some can be complex to use and their results may be difficult for beginners to understand. This creates a need for a simple and accessible tool that can identify common web and API security issues and present the findings with clear explanations and recommendations. EthioSec Scanner was developed to address this problem while focusing on authorized and responsible security testing.

3. Objectives

3.1 General Objective

To develop a simple and practical web and API vulnerability scanning platform that helps identify common security weaknesses in authorized systems and presents the findings clearly.

3.2 Specific Objectives

- To scan authorized websites and APIs for common security weaknesses.
- To identify issues like missing security headers, insecure cookies, CORS misconfigurations, and information disclosure.
- To provide clear explanations of detected security issues.
- To provide recommendations for addressing identified security issues.
- To make the security assessment process easier and more organized for users.
- To apply the cybersecurity and programming knowledge gained during the INSA training to a practical project.

4. Scope and Limitations

4.1 Scope

The project focuses on developing a web and API security assessment platform for authorized targets. EthioSec Scanner performs selected security checks, including security headers, cookies, CORS configuration, HSTS, server information disclosure, and basic website discovery. The system presents the identified issues with clear explanations and recommendations to help users understand and improve the security of their systems.

4.2 Limitations

EthioSec Scanner focuses on selected security checks and is not intended to replace a complete professional security assessment. Some security issues may require further manual testing and verification. The accuracy of the results can also depend on the target's configuration, accessibility, and the information available during the scan.

5. Methodology and System Architecture

5.1 Development Methodology

The development of EthioSec Scanner followed a step-by-step approach that included problem identification, requirements analysis, system design, implementation, integration, and testing. The team identified common web and API security concerns, defined the project's main requirements, and divided the development tasks among five team members.

Each member worked on assigned components of the system. After developing these components, the team integrated them into a unified platform. Throughout the development process, repeated testing, debugging, and feedback helped identify technical issues and improve the system. Regular communication and weekly meetings also supported coordination, progress monitoring, and collaborative problem-solving.

This iterative development approach enabled the team to apply the cybersecurity and software development knowledge gained during the INSA Cybersecurity Summer Camp to a practical project.

5.2 System Architecture

EthioSec Scanner follows a client-server architecture in which the frontend communicates with the backend through an application programming interface (API). The backend processes user requests and coordinates the appropriate scanning modules to perform selected security checks.

The system follows these main steps:

1. **Target Input:** The user enters the URL of a website or API they are authorized to assess.
2. **Request Processing:** The frontend sends the scan request to the backend through the API.
3. **Input Validation:** The backend validates the submitted information before processing the request.
4. **Security Scanning:** The backend invokes the relevant scanning modules to perform the selected security checks.
5. **Results Analysis:** The collected information is analyzed and organized into security findings.
6. **Results Presentation:** The processed findings are returned to the frontend and displayed to the user, along with explanations or recommendations where implemented.

This architecture separates the user interface from the backend processing logic, making the system easier to maintain and providing a foundation for future improvements. It also supports the integration of additional scanning capabilities and analysis features as the project evolves.

Layer	Technologies
Frontend	Vanilla HTML5, CSS3, JavaScript (ES6+), Modern Cybersecurity Theme
Backend API	Python 3.11+, FastAPI, Starlette, Uvicorn
Database	SQLAlchemy 2.0 ORM (PostgreSQL in production, SQLite for local dev)
Authentication	Argon2id password hashing, Stateless JWT (HS256) with centralized session revocation, 6-digit email verification
Scanning Engine	Asynchronous HTTP crawler, differential analysis, timing-attack verification, heuristic payload injection
AI Analysis	Multi-provider AI module (Google Gemini API support)

The general workflow of the system is:

1. The user enters an authorized target URL.
2. The system validates the provided input.
3. The backend starts the scanning process.
4. The scanner performs the selected security checks.
5. The system analyzes the collected information.
6. Detected issues are organized into findings.
7. The results and recommendations are presented to the user through the frontend.

6. System Implementation

System Implementation describes how the EthioSec Scanner was developed based on the project requirements and system design. It explains the development of the main parts of the system, including the frontend, backend, scanning modules, security checks, and results display.
The implementation includes:
Frontend: User interface for entering targets and viewing scan results.
Backend: Handles requests and controls the scanning process.
Scanner: Performs the selected security checks.
Security Checks: Analyze areas such as headers, cookies, CORS, HSTS, and server information.
Results: Organizes detected issues and presents explanations and recommendations to the user.

7. Security and Ethical Considerations

- The scanner is intended only for systems the user owns or has explicit authorization to test.
- The project should avoid aggressive or destructive testing by default.
- API keys and other secrets must be stored securely and excluded from public source repositories.
- User input must be validated to reduce abuse and backend security risks.
- Collected target data and scan evidence should be minimized and handled responsibly.
- Automated findings should not be exaggerated; false positives and false negatives are possible.
- Testing should use controlled lab systems or explicitly authorized environments.

8. Testing and Results

Test Case	Description	Result	Status
`test_registration_creates_unverified_account`	Registration initializes account with `is_verified=False` and sends 6-digit code	PASSED	VERIFIED
`test_unverified_account_cannot_access_protected_resources`	Unverified user receives 403 on `/api/targets`, `/api/scans`, and `/api/dashboard/summary`	PASSED	VERIFIED
`test_verification_activates_account_and_allows_access`	Submitting valid code sets `is_verified=True` and unblocks protected API calls	PASSED	VERIFIED
`test_invalid_verification_code_fails`	Wrong code returns 400 Bad Request	PASSED	VERIFIED
`test_verification_code_single_use`	Valid code cannot be redeemed repeatedly to reactivate account	PASSED	VERIFIED
`test_resend_cooldown_enforcement`	Second verification request within 60 seconds returns 429 Too Many Requests	PASSED	VERIFIED
`test_argon2id_hashing_for_new_accounts`	Stored password hash starts with `$argon2id$v=19$m=65536,t=3,p=4`	PASSED	VERIFIED

Test Case	Description	Result	Status
`test_password_strength_validation`	Passwords under 8 chars or over 128 chars rejected with 422	PASSED	VERIFIED
`test_account_temporary_lockout_after_failed_logins`	5 consecutive failed login attempts trigger 15-minute temporary lockout (403)	PASSED	VERIFIED
`test_session_revocation_on_logout`	`POST /api/auth/logout` revokes previous tokens (subsequent calls return 401)	PASSED	VERIFIED
`test_password_reset_workflow`	Request reset token, redeem with new password, old sessions revoked	PASSED	VERIFIED
`test_cross_user_resource_isolation`	User B cannot read or delete User A's targets or scans (IDOR protection)	PASSED	VERIFIED
`test_sensitive_credentials_never_exposed_in_api`	Responses never contain password hashes, verification hashes, or lockout fields	PASSED	VERIFIED
`test_security_headers_present`	API emits `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, CSP, etc.	PASSED	VERIFIED
`test_sql_injection_payload_safety`	Malicious SQL syntax treated as literal data; no SQL injection bypasses	PASSED	VERIFIED

9. Challenges and Solutions

Throughout the development of our project, we encountered several technical and organizational challenges that required teamwork, continuous communication, and repeated improvements.

1. Geographical Distance and Communication Challenges

One of our major challenges was that the five members of our team were located in different parts of the country. This geographical separation made effective communication, coordination, and collaboration more difficult than we initially expected.

Solution: To address this challenge, we created a Telegram group for regular communication and day-to-day discussions. We also established Friday as our fixed weekly meeting day to review progress, report completed tasks, discuss challenges, and plan the next steps. In addition, we arranged temporary meetings and conference calls whenever urgent issues required immediate discussion.

2. Integration of Different Website Components

Another significant challenge was integrating the different components of the website into a single, functional platform. Each of the five team members was responsible for a specific part of the project, including frontend development, backend development, database management, and AI-related functionality. Although dividing the work helped us develop the components independently, combining them into one system proved difficult because of differences in implementation, data flow, and compatibility.

Solution: We addressed these issues through repeated testing, debugging, coordination, and correction. The integration process required more than five rounds of revisions to ensure that the different components worked together properly. Through these iterations, we gradually improved compatibility and developed a more unified and functional system.

3. Improving Password Security

During the initial development stage, our first model accepted weak passwords without enforcing sufficient password-strength requirements. We recognized that this was a security weakness that needed to be addressed to improve the platform's security.

Solution: We improved the password validation mechanism by introducing stronger password requirements. The system was updated to accept passwords only when they met the defined strength criteria. This improvement helped us strengthen password security and reinforced the importance of incorporating security considerations into the software development process.

4. Maintaining Consistent Collaboration and Project Progress

Coordinating individual responsibilities, tracking weekly progress, and resolving technical issues remotely required consistent effort from all team members. Delays in communication or difficulties in one component could affect the integration and progress of the entire project.

Solution: We used our Telegram group for daily updates, technical discussions, and sharing feedback. Our weekly Friday meetings provided a structured opportunity to review each member's progress, identify outstanding problems, and coordinate upcoming tasks. Additional meetings were organized when urgent matters arose. This approach helped us maintain collaboration, address problems more efficiently, and keep the project moving forward.

Lessons Learned

These challenges taught us the importance of effective communication, collaborative problem-solving, iterative testing, and secure software development. We learned that building individual components successfully is only one part of software development; integrating them into a reliable, secure, and functional system requires careful coordination, continuous testing, and a shared commitment to quality.

10. Discussion

EthioSec Scanner demonstrates a practical way to combine web development and cybersecurity. Its strongest current value is the automation of repeatable passive and configuration-focused checks and the organization of results into understandable findings. The project should be evaluated on accuracy and evidence rather than on the number of vulnerability names it claims to support. Expanding into authentication, access-control, injection, or business-logic testing requires substantially more validation and must not be presented as completed unless the final implementation proves it.

11. Future Improvements

- Expand validated coverage of OWASP web and API security categories.
- Improve endpoint and JavaScript-based discovery.
- Add carefully designed authenticated assessment for explicitly authorized accounts.
- Improve evidence validation and false-positive reduction.
- Add stronger severity/risk scoring based on context and evidence.
- Generate polished downloadable assessment reports.
- Improve AI-assisted explanations while keeping scanner evidence authoritative.
- Add scan history, comparison, and team-oriented reporting where appropriate.
- Strengthen deployment hardening, secret management, logging, and rate controls.

12. Conclusion

EthioSec Scanner is a web and API security assessment project developed to apply the cybersecurity knowledge and practical skills we gained during the INSA Cybersecurity Summer Camp to a real-world software application. Our main objective was to transform what we learned into a practical tool that helps identify selected security weaknesses in authorized web applications and presents findings in a clear and understandable way. Through this project, we took an important step from learning cybersecurity concepts in a training environment toward applying them in real software development.

The current implementation focuses on selected security checks, including HTTP security headers, HTTP Strict Transport Security (HSTS), Cross-Origin Resource Sharing (CORS), cookie security attributes, server information disclosure, and basic website discovery. The platform combines frontend and backend development, database integration, automated security checks, and AI-related functionality within a unified system. These components provide a foundation for improving the accessibility and effectiveness of web security assessment.

Developing this project was not without challenges. As a team of five members working from different parts of the country, we faced difficulties in communication, coordination, and integrating independently developed components into one functional platform. We also encountered technical issues that required repeated testing, debugging, and corrections. Through regular communication, weekly meetings, collaboration, and continuous improvement, we worked to overcome these obstacles. These experiences taught us that successful software development requires not only technical knowledge but also teamwork, patience, adaptability, and a shared commitment to achieving a common goal.

However, we do not consider EthioSec Scanner a finished product or the end of our journey. We see it as the beginning of a larger and more ambitious project. Our vision is to continue developing the platform into a more comprehensive, reliable, and intelligent security assessment tool. Future development will focus on expanding validated vulnerability detection, improving evidence verification, reducing false positives, strengthening risk scoring, and making greater and more effective use of artificial intelligence. We also aim to make the system adaptable to emerging vulnerabilities, evolving attack techniques, and new web and API security risks as they arise. AI-assisted analysis will be developed to help explain findings and recommend appropriate solutions while keeping technical evidence at the center of every assessment.

Ultimately, EthioSec Scanner represents more than the completion of a cybersecurity training project. It demonstrates our effort to turn knowledge into practical solutions, learn from real development challenges, and build something that can continue to grow beyond the summer camp. We recognize that achieving our long-term vision will require further research, testing, validation, and continuous improvement. Nevertheless, the knowledge we gained, the challenges we overcame, and the foundation we established have strengthened our determination to continue this work. With continued development and responsible use, we hope EthioSec Scanner will evolve into a more capable cybersecurity platform that helps developers and organizations better understand, assess, and improve the security of their web applications and APIs.

13. References

1. OWASP Foundation. (2025). *OWASP Top 10:2025—The Ten Most Critical Web Application Security Risks.* https://top10.owasp.org/
2. OWASP Foundation. (n.d.). *Web Security Testing Guide (WSTG).* https://owasp.org/www-project-web-security-testing-guide/
3. OWASP Foundation. (n.d.). *OWASP API Security Project.* https://owasp.org/www-project-api-security/
4. OWASP Foundation. (n.d.). *OWASP Secure Headers Project.* https://owasp.org/www-project-secure-headers/
5. National Institute of Standards and Technology (NIST). (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities (NIST SP 800-218).* https://doi.org/10.6028/NIST.SP.800-218
6. Pallets Projects. (n.d.). *Flask Documentation.* https://flask.palletsprojects.com/
7. Python Software Foundation. (n.d.). *Python Documentation.* https://docs.python.org/3/
8. PostgreSQL Global Development Group. (n.d.). *PostgreSQL Documentation.* https://www.postgresql.org/docs/
9. Mozilla. (n.d.). *MDN Web Docs: HTTP Headers.* https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers
10. OWASP Foundation. (n.d.). *OWASP Cheat Sheet Series.* https://cheatsheetseries.owasp.org/